

Université Paris-Saclay

Faculté des Sciences — L3 MIAGE

# FOOTCAMP DREAMS

Application web de gestion  
d'un centre de vacances football

Rapport de Projet

## Étudiants

**Étudiant 1 :** MRABEUT Moulay Mehdi

**Étudiant 2 :** RANDRIAMANGA Iloniavo

## Informations académiques

**Formation :** L3 MIAGE

**Année universitaire :** 2025 / 2026

**Encadrant :** *Aquilina ELKHOURY*

**Date de rendu :** *09/04/2026*

# Table des matières

---

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                            | <b>3</b>  |
| 1.1      | Contexte et problématique . . . . .                            | 3         |
| 1.2      | Objectifs du projet . . . . .                                  | 3         |
| 1.3      | Plan du rapport . . . . .                                      | 3         |
| <b>2</b> | <b>Analyse des besoins</b>                                     | <b>5</b>  |
| 2.1      | Besoins fonctionnels . . . . .                                 | 5         |
| 2.1.1    | Côté client . . . . .                                          | 5         |
| 2.1.2    | Côté administrateur . . . . .                                  | 5         |
| 2.2      | Besoins non fonctionnels . . . . .                             | 6         |
| 2.3      | Cas d'utilisation principaux . . . . .                         | 6         |
| <b>3</b> | <b>Conception</b>                                              | <b>7</b>  |
| 3.1      | Architecture générale . . . . .                                | 7         |
| 3.2      | Modèle de données . . . . .                                    | 7         |
| 3.2.1    | users.json . . . . .                                           | 7         |
| 3.2.2    | reservations.json . . . . .                                    | 8         |
| 3.2.3    | rooms.json . . . . .                                           | 8         |
| 3.2.4    | activities.json . . . . .                                      | 8         |
| 3.3      | Flux utilisateur client . . . . .                              | 9         |
| 3.4      | Flux utilisateur administrateur . . . . .                      | 9         |
| 3.5      | Machine d'états des réservations . . . . .                     | 10        |
| <b>4</b> | <b>Implémentation</b>                                          | <b>11</b> |
| 4.1      | Backend PHP . . . . .                                          | 11        |
| 4.1.1    | Le fichier utils.php — Cœur du backend . . . . .               | 11        |
| 4.1.2    | Authentification . . . . .                                     | 12        |
| 4.1.3    | Gestion des réservations . . . . .                             | 13        |
| 4.1.4    | Gestion des chambres et détection des chevauchements . . . . . | 15        |
| 4.1.5    | Facturation . . . . .                                          | 15        |
| 4.1.6    | Implémentation SMTP manuelle . . . . .                         | 16        |
| 4.2      | Frontend . . . . .                                             | 17        |
| 4.2.1    | Structure des pages . . . . .                                  | 17        |
| 4.2.2    | Appels API avec fetch() et async/await . . . . .               | 17        |
| 4.2.3    | Génération du mot de passe côté client . . . . .               | 18        |
| 4.2.4    | Modal des identifiants . . . . .                               | 18        |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| 4.3      | Stockage JSON . . . . .                                   | 19        |
| 4.3.1    | Choix JSON versus base de données relationnelle . . . . . | 19        |
| 4.3.2    | Limites identifiées . . . . .                             | 19        |
| <b>5</b> | <b>Fonctionnalités restantes et perspectives</b>          | <b>21</b> |
| 5.1      | Fonctionnalités restantes . . . . .                       | 21        |
| 5.1.1    | Dashboard client à compléter . . . . .                    | 21        |
| 5.1.2    | Authentification par sessions PHP . . . . .               | 21        |
| 5.1.3    | Sécurisation des endpoints API . . . . .                  | 21        |
| 5.2      | Perspectives d'amélioration . . . . .                     | 21        |
| <b>6</b> | <b>Difficultés rencontrées</b>                            | <b>23</b> |
| 6.1      | Implémentation manuelle du protocole SMTP . . . . .       | 23        |
| 6.2      | Gestion de la concurrence sur les fichiers JSON . . . . . | 23        |
| 6.3      | Détection des chevauchements de réservations . . . . .    | 23        |
| 6.4      | Migration des mots de passe legacy . . . . .              | 23        |
| <b>7</b> | <b>Conclusion</b>                                         | <b>24</b> |
| 7.1      | Bilan des réalisations . . . . .                          | 24        |
| 7.2      | Compétences acquises . . . . .                            | 24        |
| 7.3      | Perspectives . . . . .                                    | 25        |

# 1. Introduction

---

## 1.1 Contexte et problématique

Dans le cadre de notre troisième année de Licence MIAGE à l'Université Paris-Saclay, nous avons été amenés à concevoir et développer une application web complète répondant à un besoin concret de gestion. Le projet **FootCamp Dreams** s'inscrit dans ce contexte : il s'agit d'une plateforme web dédiée à la gestion d'un centre de vacances spécialisé dans le football.

Ce type d'établissement fait face à plusieurs défis organisationnels : la gestion manuelle des réservations est chronophage, sujette aux erreurs et peu adaptée aux attentes des clients modernes qui souhaitent pouvoir effectuer leurs démarches en ligne. La nécessité de disposer d'un outil numérique centralisé, accessible à la fois par les clients et par l'équipe administrative, est donc évidente.

## 1.2 Objectifs du projet

Le projet **FootCamp Dreams** a pour objectifs principaux :

- Permettre aux clients de consulter les offres du centre et de soumettre une demande de réservation en ligne, en choisissant leurs dates, le type de chambre et les activités sportives souhaitées.
- Offrir à l'équipe administrative un tableau de bord complet permettant de visualiser toutes les réservations, de les valider ou rejeter, d'assigner des chambres et d'envoyer les factures.
- Automatiser la communication par email : confirmation de réservation, envoi des identifiants de connexion, facture.
- Fournir à chaque client un espace personnel pour consulter l'état de ses réservations.

## 1.3 Plan du rapport

Ce rapport est organisé de la manière suivante. Le chapitre 2 présente l'analyse des besoins fonctionnels et non fonctionnels du projet. Le chapitre 3 détaille les choix de conception : architecture technique, modèle de données et flux utilisateurs. Le chapitre 4 décrit l'implémentation concrète, avec des extraits de code commentés. Le chapitre 5 présente les fonctionnalités restantes à implémenter ainsi que les perspectives

d'amélioration. Le chapitre 6 dresse le bilan des difficultés rencontrées. Enfin, le chapitre 7 conclut le rapport.

## 2. Analyse des besoins

---

### 2.1 Besoins fonctionnels

#### 2.1.1 Côté client

- **Consultation** : Le client peut consulter la landing page présentant le centre, ses services, ses statistiques et sa galerie d'images.
- **Réservation** : Le client peut soumettre une demande de réservation en renseignant ses informations personnelles, ses dates d'arrivée et de départ, le nombre de participants, les activités souhaitées et le type de chambre désiré.
- **Confirmation** : Après soumission, le client reçoit un email de confirmation de réception de sa demande.
- **Identifiants** : Lors de la validation par l'admin, le client reçoit ses identifiants de connexion par email.
- **Espace personnel** : Le client peut se connecter et consulter l'état de ses réservations depuis son tableau de bord.

#### 2.1.2 Côté administrateur

- **Tableau de bord** : L'administrateur dispose d'une vue complète sur toutes les réservations et leur statut.
- **Gestion des chambres** : Visualisation de la disponibilité de l'ensemble des chambres.
- **Validation / Rejet** : L'administrateur peut valider ou rejeter une réservation en attente.
- **Assignment** : Lors de la validation, une chambre est assignée à la réservation et un compte client est créé automatiquement si nécessaire.
- **Facturation** : L'administrateur peut envoyer la facture détaillée au client par email.
- **Logs emails** : Consultation de l'historique de tous les emails envoyés.
- **Configuration** : L'administrateur peut configurer les paramètres SMTP depuis l'interface.

## 2.2 Besoins non fonctionnels

| Catégorie      | Description                                                                                                          |
|----------------|----------------------------------------------------------------------------------------------------------------------|
| Sécurité       | Les mots de passe sont hachés avec bcrypt. Les données utilisateur sont validées côté serveur avant tout traitement. |
| Disponibilité  | L'application doit fonctionner sur tout navigateur moderne (Chrome, Firefox, Safari, Edge).                          |
| Responsive     | L'interface s'adapte aux écrans mobiles, tablettes et ordinateurs grâce à Bootstrap 5.3 et des breakpoints CSS.      |
| Performance    | Les images sont générées en SVG inline. Le temps de chargement visé est inférieur à 2 secondes.                      |
| Maintenabilité | Le code backend est centralisé dans <code>utils.php</code> . Chaque endpoint a une responsabilité unique.            |
| Portabilité    | Aucune dépendance à une base de données externe : le stockage JSON facilite le déploiement.                          |

TABLE 2.1 – Besoins non fonctionnels du projet

## 2.3 Cas d'utilisation principaux

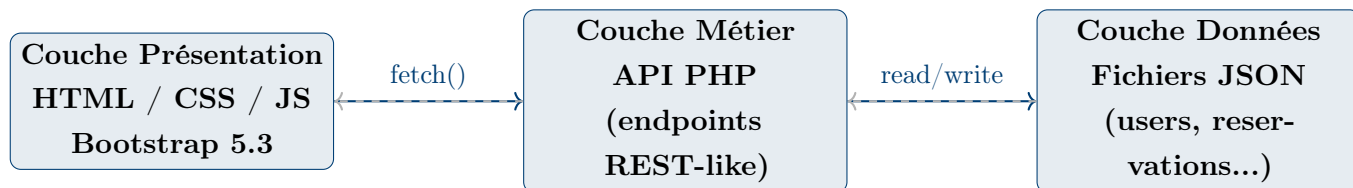
| Acteur  | Cas d'utilisation          | Description                                     |
|---------|----------------------------|-------------------------------------------------|
| Client  | Soumettre une réservation  | Remplir et envoyer le formulaire de réservation |
| Client  | Se connecter               | Accéder au tableau de bord client               |
| Client  | Consulter ses réservations | Voir le statut de ses demandes                  |
| Admin   | Se connecter               | Accéder au tableau de bord admin                |
| Admin   | Valider une réservation    | Accepter et assigner une chambre                |
| Admin   | Rejeter une réservation    | Refuser une demande                             |
| Admin   | Envoyer une facture        | Transmettre la facture par email                |
| Admin   | Consulter les emails       | Voir l'historique des envois                    |
| Système | Envoyer un email           | Confirmation, identifiants, facture             |

TABLE 2.2 – Cas d'utilisation principaux

## 3. Conception

### 3.1 Architecture générale

L'application repose sur une architecture trois tiers classique :



- **Couche présentation** : Trois pages HTML principales (`index.html`, `client-dashboard.html`, `admin-dashboard.html`) accompagnées de JavaScript vanilla et jQuery pour les interactions et les appels AJAX.
- **Couche métier** : Un ensemble de scripts PHP organisés par domaine fonctionnel (auth, reservations, rooms, billing, email). Le fichier `api/utills.php` centralise toutes les fonctions utilitaires partagées.
- **Couche données** : Des fichiers JSON stockés dans le répertoire `data/`, sans recours à un système de gestion de base de données relationnelle.

### 3.2 Modèle de données

#### 3.2.1 users.json

| Champ     | Type   | Description                                                       |
|-----------|--------|-------------------------------------------------------------------|
| id        | entier | Identifiant unique auto-incrémenté                                |
| nom       | chaîne | Nom de famille                                                    |
| prenom    | chaîne | Prénom                                                            |
| email     | chaîne | Adresse email (clé fonctionnelle unique)                          |
| password  | chaîne | Mot de passe haché en bcrypt                                      |
| telephone | chaîne | Numéro de téléphone (10 chiffres)                                 |
| role      | chaîne | Rôle de l'utilisateur : <code>client</code> ou <code>admin</code> |

TABLE 3.1 – Structure de `users.json`



### 3.2.2 reservations.json

| Champ            | Type     | Description                                          |
|------------------|----------|------------------------------------------------------|
| id               | entier   | Identifiant unique                                   |
| nom, prenom      | chaîne   | Informations du client                               |
| email, telephone | chaîne   | Coordonnées                                          |
| date_arrivee     | date     | Date d'arrivée (YYYY-MM-DD)                          |
| date_depart      | date     | Date de départ (YYYY-MM-DD)                          |
| nb_personnes     | entier   | Nombre de participants                               |
| activities       | tableau  | Liste des IDs d'activités choisies                   |
| selected_rooms   | objet    | Nombre de chambres par type choisi                   |
| status           | chaîne   | <b>en_attente</b> , <b>validee</b> ou <b>rejetee</b> |
| deposit          | entier   | Acompte versé (80€ par défaut)                       |
| room_numbers     | tableau  | Numéros de chambres assignées                        |
| created_at       | datetime | Date de création de la demande                       |

TABLE 3.2 – Structure de reservations.json

### 3.2.3 rooms.json

Le fichier contient deux clés principales :

- **types** : liste des types de chambres avec leurs caractéristiques.
- **rooms** : liste de toutes les chambres individuelles numérotées.

| Type   | Capacité    | Prix/nuît | Nombre total |
|--------|-------------|-----------|--------------|
| Simple | 1 personne  | 50 €      | 100 chambres |
| Double | 2 personnes | 90 €      | 50 chambres  |
| Triple | 3 personnes | 120 €     | 50 chambres  |

TABLE 3.3 – Types de chambres disponibles

### 3.2.4 activities.json

| Activité               | Prix  | Min. participants | Max. participants |
|------------------------|-------|-------------------|-------------------|
| Entraînement collectif | 50 €  | 22                | —                 |
| Match amical           | 75 €  | 15                | —                 |
| Cours privé            | 100 € | —                 | 5                 |
| Tournoi                | 200 € | 15                | —                 |

TABLE 3.4 – Activités proposées

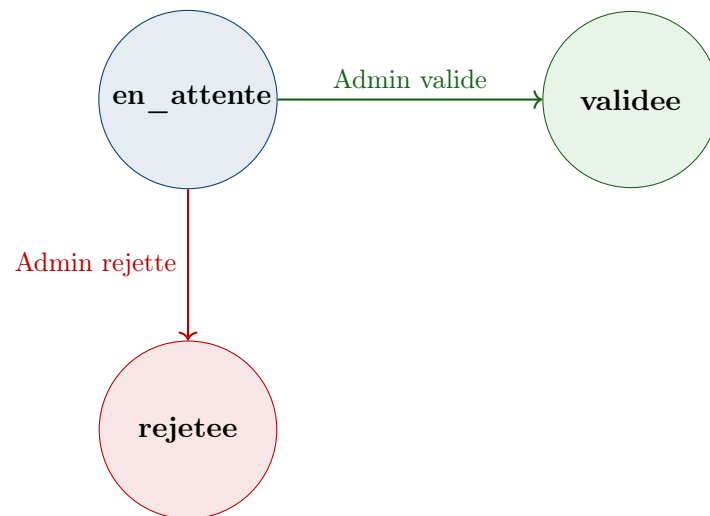
### 3.3 Flux utilisateur client

1. Le client visite `index.html` et consulte la landing page.
2. Il remplit le formulaire de réservation (informations personnelles, dates, activités, type de chambre).
3. Il clique sur « Soumettre ma réservation » : un appel AJAX envoie les données à `api/reservations/reservation_request.php`.
4. Un email de confirmation est envoyé automatiquement à son adresse.
5. L'administrateur examine la demande et la valide ou la rejette.
6. En cas de validation, le client reçoit ses identifiants de connexion par email.
7. Il se connecte sur `client-dashboard.html` et consulte le statut de sa réservation.

### 3.4 Flux utilisateur administrateur

1. L'administrateur accède à `admin-dashboard.html` et se connecte.
2. Il visualise la liste complète des réservations et la disponibilité des chambres.
3. Pour chaque réservation en attente, il peut consulter les détails et choisir de valider ou rejeter.
4. **En cas de validation** : le système vérifie la disponibilité des chambres, assigne les chambres requises, crée un compte client si nécessaire, génère un mot de passe, et envoie un email avec les identifiants.
5. **En cas de rejet** : le statut est mis à jour et le client peut être notifié.
6. L'administrateur peut envoyer la facture détaillée à tout moment depuis le tableau de bord.

### 3.5 Machine d'états des réservations



Une réservation commence toujours à l'état **en\_attente**. Elle ne peut transitionner que vers **validee** ou **rejetee**, et ces états sont terminaux : une réservation déjà traitée ne peut plus être modifiée.

## 4. Implémentation

---

### 4.1 Backend PHP

#### 4.1.1 Le fichier utils.php — Cœur du backend

Le fichier `api/utils.php` est inclus par tous les endpoints PHP via `require_once`. Il regroupe l'ensemble des fonctions utilitaires partagées, ce qui évite la duplication de code et centralise la logique d'accès aux données.

```
1 function readJson($fileName) {
2     $path = __DIR__ . '/../data/' . $fileName;
3     if (!file_exists($path)) {
4         return [];
5     }
6     $content = file_get_contents($path);
7     $data = json_decode($content, true);
8     return is_array($data) ? $data : [];
9 }
10
11 function writeJson($fileName, $data) {
12     $path = __DIR__ . '/../data/' . $fileName;
13     file_put_contents(
14         $path,
15         json_encode($data, JSON_PRETTY_PRINT |
16             JSON_UNESCAPED_UNICODE)
17     );
18 }
19
20 function jsonResponse($data) {
21     header('Content-Type: application/json; charset=utf-8');
22     echo json_encode($data, JSON_UNESCAPED_UNICODE);
23     exit;
24 }
25
26 // Genere le prochain ID unique en prenant le max existant + 1
27 function nextId($items) {
28     if (empty($items)) return 1;
29     $ids = array_column($items, 'id');
30     return max($ids) + 1;
31 }
```

Listing 4.1 – Fonctions de lecture/écriture JSON (utils.php)

La fonction `nextId` garantit un identifiant unique même en cas de suppressions

intermédiaires dans le tableau, car elle se base sur le maximum existant et non sur la taille du tableau.

### 4.1.2 Authentification

#### Inscription (register.php)

Lors de l'inscription, le mot de passe est haché avec l'algorithme **bcrypt** via la fonction native PHP `password_hash`. Bcrypt est préféré à MD5 ou SHA-1 car il est intentionnellement lent et intègre un sel aléatoire, rendant les attaques par force brute et par table arc-en-ciel très difficiles.

```
1 $newUser = [  
2     'id'          => nextId($users),  
3     'nom'         => $nom,  
4     'prenom'     => $prenom,  
5     'email'      => $email,  
6     // PASSWORD_DEFAULT correspond a bcrypt  
7     'password'   => password_hash($password, PASSWORD_DEFAULT),  
8     'telephone' => $telephone,  
9     'role'       => 'client'  
10 ];  
11 $users[] = $newUser;  
12 writeJson('users.json', $users);
```

Listing 4.2 – Hachage du mot de passe à l'inscription (register.php)

#### Connexion (login.php)

La connexion vérifie le mot de passe via `password_verify()`. Le fichier gère également une migration transparente des anciens mots de passe stockés en clair vers le format haché, assurant la compatibilité ascendante.

```
1 foreach ($users as &$user) {  
2     if ($user['email'] === $email) {  
3         // Cas 1 : mot de passe hache (cas normal)  
4         if (password_verify($password, $user['password'])) {  
5             jsonResponse(['success' => true, 'user' => $user]);  
6         }  
7         // Cas 2 : mot de passe en clair (legacy) -> migration  
8         if ($user['password'] === $password) {  
9             $user['password'] = password_hash($password,  
10            PASSWORD_DEFAULT);  
11            writeJson('users.json', $users);  
12            jsonResponse(['success' => true, 'user' => $user]);  
13        }  
14    }
```

```
14 }
15 jsonResponse(['success' => false, 'message' => 'Identifiants
    incorrects']);
```

Listing 4.3 – Connexion avec migration legacy (login.php)

### 4.1.3 Gestion des réservations

#### Création d'une demande (reservation\_request.php)

L'endpoint lit les données depuis le body de la requête HTTP via `php://input` plutôt que `$_POST`, car le frontend envoie du JSON brut avec le Content-Type `application/json`.

```
1 $input = json_decode(file_get_contents('php://input'), true) ?? [];
2
3 // Validation du format telephone (10 chiffres)
4 if (!preg_match('/^[0-9]{10}$/', $telephone)) {
5     jsonResponse(['success' => false,
6         'message' => 'Telephone invalide (10 chiffres
7         requis)']);
8 }
9
10 // Validation de la coherence des dates
11 $arrival = strtotime($date_arrivee);
12 $departure = strtotime($date_depart);
13 if (!$arrival || !$departure || $arrival >= $departure) {
14     jsonResponse(['success' => false,
15         'message' => 'Les dates sont invalides (arrivee <
16         depart)']);
17 }
18
19 $newReservation = [
20     'id' => nextId($reservations),
21     'status' => 'en_attente',
22     'deposit' => 80,
23     'created_at' => date('Y-m-d H:i:s'),
24     // ... autres champs
25 ];
```

Listing 4.4 – Lecture du body JSON et validation (reservation\_request.php)

#### Validation par l'administrateur (admin\_validate\_reservation.php)

Lors de la validation, le système vérifie la disponibilité des chambres, crée le compte client si nécessaire et envoie les identifiants par email.

```

1 $alreadyExists = false;
2 foreach ($users as $user) {
3     if ($user['email'] === $reservation['email']) {
4         $alreadyExists = true;
5         break;
6     }
7 }
8
9 if (!$alreadyExists) {
10     // Generer un mot de passe : prenom en minuscules + "2026"
11     $plainPassword = generatePassword($reservation['prenom']);
12     $users[] = [
13         'id'          => nextId($users),
14         'email'       => $reservation['email'],
15         'password'    => password_hash($plainPassword,
16         PASSWORD_DEFAULT),
17         'role'        => 'client'
18     ];
19     $passwordToSend = $plainPassword;
20 } else {
21     $passwordToSend = null; // Le client a deja un compte
22 }
23
24 $reservation['status'] = 'validee';
25 writeJson('reservations.json', $reservations);
26 writeJson('users.json', $users);

```

Listing 4.5 – Creation du compte client a la validation

## Rejet d'une réservation (admin\_reject\_reservation.php)

Le rejet est simple : on vérifie que la réservation est bien en attente, puis on change son statut.

```

1 foreach ($reservations as &$reservation) {
2     if ($reservation['id'] === $id) {
3         if ($reservation['status'] !== 'en_attente') {
4             jsonResponse(['success' => false,
5                 'message' => 'Cette reservation a deja
6                 ete traitee.']);
7         }
8         $reservation['status'] = 'rejetee';
9         writeJson('reservations.json', $reservations);
10        jsonResponse(['success' => true, 'message' => 'Reservation
11        rejetee.']);
12    }
13 }

```

## Listing 4.6 – Rejet d’une réservation

## 4.1.4 Gestion des chambres et détection des chevauchements

La détection des chevauchements de réservations est un point algorithmique central. Deux périodes  $[A, B]$  et  $[C, D]$  se chevauchent si et seulement si  $A < D$  et  $C < B$ , ce qui équivaut à : elles *ne se chevauchent pas* si  $B \leq C$  ou  $A \geq D$ .

```

1 foreach ($typeRooms as $room) {
2     $isAvailable = true;
3     foreach ($reservations as $res) {
4         if ($res['status'] === 'validee'
5             && isset($res['room_number'])
6             && $res['room_number'] == $room['number']) {
7
8             $resStart = strtotime($res['date_arrivee']);
9             $resEnd    = strtotime($res['date_depart']);
10
11             // Chevauchement : NOT (depart_new <= debut_res OR
12             arrivee_new >= fin_res)
13             if (!($departure <= $resStart || $arrival >= $resEnd))
14         {
15                 $isAvailable = false;
16                 break;
17             }
18         }
19         if ($isAvailable) {
20             $availableRoomNumbers[] = $room['number'];
21         }
22     }
23 }

```

Listing 4.7 – Detection des chevauchements de dates (get\_available\_rooms.php)

## 4.1.5 Facturation

La facture est calculée dynamiquement à partir des données de la réservation :

$$Total = \underbrace{(prix\_nuit \times nb\_nuits)}_{hbergement} + \underbrace{\sum prix\_activits}_{activits} - \underbrace{dpt}_{acompte(80)}$$

```

1 $nights    = ($departure - $arrival) / (24 * 3600);
2 $roomPrice = $room['price_per_night'] * $nights;
3

```



```
4 $activitiesCost = 0;
5 foreach ($reservation['activities'] as $actId) {
6     foreach ($activities as $act) {
7         if ($act['id'] === $actId) {
8             $activitiesCost += $act['price'];
9         }
10    }
11 }
12
13 $subtotal = $roomPrice + $activitiesCost;
14 $totalDue = $subtotal - $reservation['deposit'];
```

Listing 4.8 – Calcul de la facture (get\_invoice.php)

### 4.1.6 Implémentation SMTP manuelle

L'envoi d'emails est réalisé sans librairie externe, en implémentant manuellement le protocole SMTP via des sockets TCP PHP. Cette approche, bien que plus complexe, permet de comprendre en profondeur le fonctionnement du protocole.

La séquence SMTP suivie est la suivante :

1. Ouverture d'une connexion TCP vers smtp.gmail.com:587
2. Réception du code 220 (serveur prêt)
3. Envoi de EHLO localhost → réponse 250
4. Envoi de STARTTLS → réponse 220 → activation du chiffrement TLS
5. Renvoi de EHLO localhost après TLS → 250
6. Envoi de AUTH LOGIN → réponse 334
7. Envoi du login en Base64 → réponse 334
8. Envoi du mot de passe en Base64 → réponse 235 (authentifié)
9. MAIL FROM, RCPT TO, DATA, contenu, QUIT

```
1 // Ouverture socket TCP
2 $socket = @stream_socket_client(
3     'tcp://' . SMTP_HOST . ':' . SMTP_PORT, $errno, $errstr, 30
4 );
5
6 // Activation TLS
7 $writeCommand('STARTTLS');
8 $response = $readResponse();
9 stream_socket_enable_crypto($socket, true,
10     STREAM_CRYPTO_METHOD_TLS_CLIENT);
11
12 // Authentification en Base64
```

```
12 $writeCommand('AUTH LOGIN');
13 $writeCommand(base64_encode(SMTP_USER));
14 $writeCommand(base64_encode(SMTP_PASSWORD));
15 // Code attendu : 235 (authentication successful)
```

Listing 4.9 – Extrait de la séquence SMTP (utils.php)

Tous les emails envoyés sont systématiquement enregistrés dans `data/email_logs.json` via la fonction `logEmail()`, assurant une traçabilité complète.

## 4.2 Frontend

### 4.2.1 Structure des pages

Le projet comporte trois pages HTML principales :

- **index.html** : Landing page avec hero section, 6 feature cards, galerie SVG, statistiques, formulaire de réservation et portails de connexion client/admin.
- **client-dashboard.html** : Espace personnel du client pour consulter ses réservations.
- **admin-dashboard.html** : Tableau de bord complet de l'administrateur.

### 4.2.2 Appels API avec `fetch()` et `async/await`

Les interactions avec le backend utilisent l'API `fetch()` moderne avec la syntaxe `async/await`, qui rend le code asynchrone lisible comme du code synchrone et facilite la gestion des erreurs.

```
1 async function submitReservation(formData) {
2   try {
3     const response = await fetch('api/reservations/
4 reservation_request.php', {
5       method: 'POST',
6       headers: { 'Content-Type': 'application/json' },
7       body: JSON.stringify(formData)
8     });
9     const data = await response.json();
10
11     if (data.success) {
12       showCredentialsModal(formData.email, data.password,
13 data.reservation_id);
14     } else {
15       showMsg(msgDiv, data.message, false);
16     }
17   } catch (error) {
```

```
16     showMsg(msgDiv, 'Erreur reseau. Veuillez reessayer.', false
17   );
18 }
```

Listing 4.10 – Soumission du formulaire de réservation (JS)

### 4.2.3 Génération du mot de passe côté client

La page d'accueil dispose d'une fonction JavaScript de génération de mot de passe fort, utilisée pour la démonstration dans le modal.

```
1  function generatePassword() {
2    const upper    = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
3    const lower    = 'abcdefghijklmnopqrstuvwxyz';
4    const numbers  = '0123456789';
5    const special  = '!@#$%^&*';
6    const all      = upper + lower + numbers + special;
7
8    let password = '';
9    // Garantir au moins un caractere de chaque categorie
10   password += upper[Math.floor(Math.random() * upper.length)];
11   password += numbers[Math.floor(Math.random() * numbers.length)
12 ];
13   password += special[Math.floor(Math.random() * special.length)
14 ];
15
16   // Completer jusqu'a 12 caracteres
17   for (let i = 3; i < 12; i++) {
18     password += all[Math.floor(Math.random() * all.length)];
19   }
20   return password.split('').sort(() => Math.random() - 0.5).join
21   ('');
22 }
```

Listing 4.11 – Génération de mot de passe côté client (index.html)

### 4.2.4 Modal des identifiants

Après validation d'une réservation, un modal s'affiche pour présenter à l'utilisateur ses identifiants de connexion avec des boutons de copie dans le presse-papier.

```
1  async function copyToClipboard(text, btn) {
2    try {
3      await navigator.clipboard.writeText(text);
4      btn.textContent = 'Copie !';
5      setTimeout(() => btn.textContent = 'Copier', 2000);
6    } catch (err) {
7      console.log('Erreur lors de la copie');
8    }
9  }
```

```

6      } catch (err) {
7          // Fallback pour les navigateurs ne supportant pas
Clipboard API
8          const ta = document.createElement('textarea');
9          ta.value = text;
10         document.body.appendChild(ta);
11         ta.select();
12         document.execCommand('copy');
13         document.body.removeChild(ta);
14     }
15 }

```

Listing 4.12 – Copie dans le presse-papier (assets/client.js)

## 4.3 Stockage JSON

### 4.3.1 Choix JSON versus base de données relationnelle

| Critère          | JSON (choix actuel)                                              | MySQL (alternative)        |
|------------------|------------------------------------------------------------------|----------------------------|
| Déploiement      | Aucune configuration requise                                     | Nécessite un serveur MySQL |
| Lecture/Écriture | <code>file_get_contents</code><br><code>file_put_contents</code> | / Requetes SQL avec PDO    |
| Concurrence      | Risque de corruption simultanée                                  | Transactions ACID          |
| Recherche        | Parcours linéaire $O(n)$                                         | Index, requêtes optimisées |
| Relations        | Manuelles (jointures en PHP)                                     | Clés étrangères natives    |
| Adapté à         | Prototype, faible charge                                         | Production, forte charge   |

TABLE 4.1 – Comparaison JSON vs MySQL

### 4.3.2 Limites identifiées

### Limites du stockage JSON

- **Concurrence** : si deux requêtes écrivent simultanément dans le même fichier, l'une peut écraser l'autre (*race condition*).
- **Performance** : les recherches nécessitent de parcourir l'intégralité du tableau. Sur un grand volume de données, cela devient lent.
- **Intégrité** : aucune contrainte de clé étrangère n'est vérifiée automatiquement.
- **Scalabilité** : le modèle JSON n'est pas adapté à une montée en charge significative.

## 5. Fonctionnalités restantes et perspectives

---

### 5.1 Fonctionnalités restantes

#### 5.1.1 Dashboard client à compléter

L'interface côté client est partiellement implémentée. Les fonctionnalités suivantes restent à développer :

- Affichage détaillé de chaque réservation avec toutes ses informations.
- Consultation de la facture directement depuis le dashboard.
- Possibilité d'effectuer une nouvelle réservation depuis l'espace client.
- Affichage des activités planifiées pour le séjour.

#### 5.1.2 Authentification par sessions PHP

Actuellement, les données de l'utilisateur connecté sont stockées côté client via `localStorage`. Cette approche présente un risque de sécurité : un utilisateur malveillant pourrait modifier les données stockées dans le navigateur. Il conviendrait d'implémenter des sessions PHP côté serveur (`session_start()`, `$_SESSION`) pour maintenir l'état d'authentification de manière fiable.

#### 5.1.3 Sécurisation des endpoints API

Les endpoints admin sont actuellement accessibles sans vérification d'authentification côté serveur. Il faudrait ajouter un middleware de vérification de session ou un mécanisme de tokens (par exemple JWT) pour protéger ces routes.

### 5.2 Perspectives d'amélioration

- **Migration vers MySQL** : remplacement des fichiers JSON par une base de données relationnelle, avec des requêtes préparées via PDO pour prévenir les injections SQL.
- **Authentification JWT** : implémentation de JSON Web Tokens pour une authentification stateless et sécurisée.
- **Variables d'environnement** : déplacement des identifiants SMTP dans un fichier `.env` (non versionné) plutôt que dans `config.php`.

- **Génération de mot de passe** : remplacement de `prenom + "2026"` par un générateur cryptographique en PHP (`bin2hex(random_bytes(8))`).
- **Notifications temps réel** : utilisation de WebSockets ou de Server-Sent Events pour notifier l'admin en temps réel des nouvelles réservations.
- **Paieement en ligne** : intégration d'une API de paiement (Stripe, PayPal) pour encaisser l'acompte directement à la réservation.

## 6. Difficultés rencontrées

---

### 6.1 Implémentation manuelle du protocole SMTP

L'une des difficultés majeures du projet a été l'implémentation du protocole SMTP sans recourir à une librairie externe. Il a fallu comprendre précisément la séquence des commandes et des codes de réponse, gérer l'activation de TLS via **STARTTLS**, et encoder correctement les identifiants en Base64. Les erreurs de connexion SMTP étaient difficiles à déboguer car le serveur Gmail impose des contraintes strictes (authentification à deux facteurs, mots de passe d'application).

### 6.2 Gestion de la concurrence sur les fichiers JSON

L'utilisation de fichiers JSON comme stockage entraîne un risque de *race condition* lors d'écritures simultanées. Ce problème a été partiellement atténué en minimisant la durée entre la lecture et l'écriture d'un fichier, mais une solution robuste (verrouillage de fichier avec `flock()` ou migration vers une base de données) n'a pas encore été implementée.

### 6.3 Détection des chevauchements de réservations

La logique de détection des conflits de dates a nécessité une attention particulière. La formule correcte de non-chevauchement entre deux intervalles  $[A, B]$  et  $[C, D]$  est  $B \leq C$  ou  $A \geq D$ , soit par négation : il y a chevauchement si  $A < D$  et  $C < B$ . Des erreurs de logique initiales (utilisation de  $\leq$  au lieu de  $<$ ) entraînaient des faux positifs lors de réservations consécutives (départ le jour J = arrivée le jour J).

### 6.4 Migration des mots de passe legacy

En cours de développement, le format de stockage des mots de passe a évolué du texte clair vers le hachage bcrypt. Il a fallu implémenter une migration transparente dans `login.php`, capable de détecter si un mot de passe est encore en clair, de le vérifier directement, puis de le remplacer par sa version hachée de manière automatique et invisible pour l'utilisateur.



## 7. Conclusion

---

### 7.1 Bilan des réalisations

Ce projet nous a permis de concevoir et développer une application web complète de gestion d'un centre de vacances football. Les fonctionnalités principales sont opérationnelles : la landing page, le formulaire de réservation avec validation des données, le tableau de bord administrateur avec gestion complète des réservations (validation, rejet, assignation de chambres), la facturation et l'envoi automatique d'emails.

| Fonctionnalité                      | Statut    |
|-------------------------------------|-----------|
| Landing page                        | Réalisée  |
| Formulaire de réservation           | Réalisée  |
| Dashboard administrateur            | Réalisée  |
| Validation / Rejet des réservations | Réalisée  |
| Gestion des chambres                | Réalisée  |
| Envoi d'emails (SMTP manuel)        | Réalisée  |
| Facturation                         | Réalisée  |
| Dashboard client complet            | Partielle |
| Sessions PHP côté serveur           | À faire   |
| Sécurisation des endpoints admin    | À faire   |

TABLE 7.1 – Bilan des fonctionnalités

### 7.2 Compétences acquises

Ce projet nous a permis de renforcer et d'acquérir plusieurs compétences techniques et organisationnelles :

- Conception d'une architecture web three-tier sans framework.
- Implémentation d'une API REST-like en PHP.
- Maîtrise du protocole SMTP et de son implémentation bas niveau.
- Gestion de la sécurité des mots de passe avec bcrypt.
- Utilisation avancée de JavaScript moderne (fetch, async/await, Clipboard API).
- Conception d'interfaces responsives avec Bootstrap 5.
- Gestion de projet en binôme et organisation du code.

## 7.3 Perspectives

FootCamp Dreams constitue une base solide et fonctionnelle. Les évolutions prioritaires identifiées sont la migration vers une base de données MySQL pour la robustesse, l'implémentation de sessions PHP pour sécuriser l'authentification, et le développement complet du dashboard client. À plus long terme, l'intégration d'un système de paiement en ligne et de notifications en temps réel permettrait d'offrir une expérience utilisateur complète et professionnelle.

*Rapport rédigé par MRABEUT Moulay Mehdi et RANDRIAMANGA Itoniavo  
L3 MIAGE — Université Paris-Saclay — 2025/2026*